

ECN-207 Questions from 2022

- (8 points) Given below data, find the time that a request servicing completion time for both open-row policy and close-row policy. Hit latency = 20; Row address strobe = 30; Precharge latency = 50.

ID	Row,Column	TimeOfArrival
1	40,23	0
2	41,34	10
3	100,33	90
4	41,12	150
5	41,11	180
6	41,10	260
7	40,9	300
8	41,10	340

Column information is immaterial here! Hit latency is same as read-write latency. 20+30 will be incurred if row-buffer miss happens. 20+30+50 will be incurred if row-buffer conflict happens. Here, every correct answer fetches 0.5 marks.

ID	Row,Column	TimeOfArrival	OpenRow	CloseRow
1	40,23	0	50	50
2	41,34	10	150	150
3	100,33	90	250	250
4	41,12	150	350	350
5	41,11	180	370	450
6	41,10	260	390	550
7	40,9	300	490	650
8	41,10	340	590	750

- (3 points) A DRAM uses open-row policy. For Figure 2, find the stall time of appA and appB separately. Assume FR-FCFS policy. Your answer can be an expression/function in terms of H (hitLatency), M (missLatency) and C (conflictLatency).

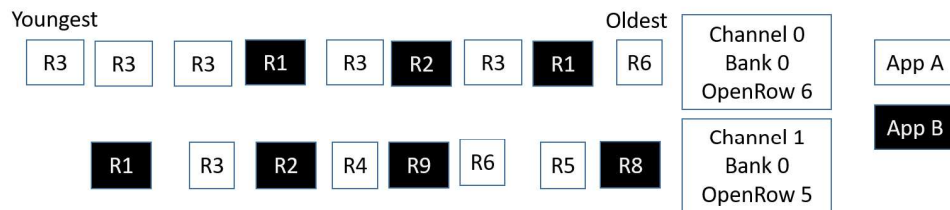


Figure 1: DRAM scheduling

FRFCFS will reschedule in following way:

AppA on Channel 0: $6H+2C$ (App1 is finished as soon as all R3 are serviced)

AppA on Channel 1: $1H+6C$ (App1 is finished as soon as the last R3 is serviced)

So, answer: $\max(6H+2C, 1H+6C)$. 1.5 marks. If someone wrote the answer as $1H+6C$, that is also taken as correct.

AppB on Channel 0: $6H+3C$

AppB on Channel 1: $1H+7C$

So, answer: $\max(6H+3C, 1H+7C)$. If someone wrote the answer as $1H+7C$, that is also taken as correct. 1.5 marks

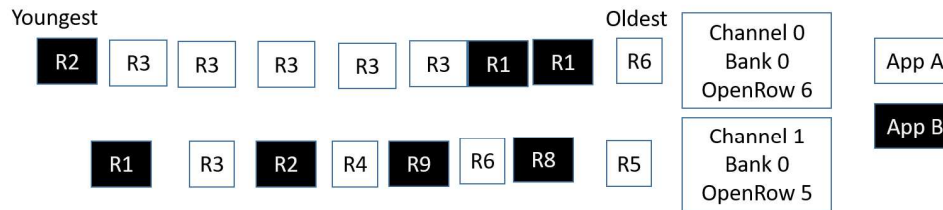


Figure 2: DRAM scheduling

3. (3 points) Assume that a processor uses a page size of 16 KB. Assume that the L1 cache must be 4-way set-associative and has a block size of 64B. You're implementing a virtually indexed physically tagged L1 cache, such that the TLB access is not required for indexing into the L1 cache. What is the largest L1 cache that you can design (unit in B or KB or MB should be shown mandatorily)?

In this case, the maximum number of sets in L1 is equal to $(16 \times 1024) / 64 = 256$

Number of ways = 4

So, largest cache size = $256 * 4 * 64B = 64KB$

4. (2 points) A processor with 32 bit physical address has only L1 cache. L1 cache has 32B block size, has 4 ways and a total size of 16KB. The processor uses a page size of 1KB. How many page colors are there in the system?

Number of sets * block size = $16KB / 4 = 4KB$

The page colors = $4KB / 1KB = 4$

5. (1 point) If a system has 512B page size, physical address space of 4KB and virtual address space of 8KB, find number of bits in (a) virtual page number and (b) physical frame number.

(a) 4 (b) 3

6. (2 points) We have two processes sharing a processor (VPN= virtual page number, PFN = physical frame number).

For process 0, VPN to PFN mappings are: [0, 1], [4, 7], [7, 2], [14, 3].

For process 1, VPN to PFN mappings are: [0, 2], [5, 7], [8, 21], [13, 11].

If the page size is 8B and we use VIPT design, then find a physical address whose data can exist in two different locations in the cache.

To find any one physical address, we can set the page offset to be any value between 000 to 111. I am showing solution with 000.

Any byte in frame 7 is seen as 2 different virtual pages from processes 0, 1. Physical address of the first byte in frame 7 = 111000. This is the answer.

Similarly, any byte in frame 2 is seen as two different virtual pages from processes 0, 1. Physical address of the first byte in frame 2 = 010000. This is also correct answer.

If someone has put more zeros in the beginning, it does not matter.

7. (4 points) An array $X[9][20]$ is stored in the memory with each element requiring 3 bytes of storage. For example, $X[9][20]$ means valid row indices are 0 to 8 and valid column indices are 0 to 19. The indexing starts from 0 (as done in C/C++). You can leave answer as $p * q + r$.

If the base address of array is 3000, calculate the location of $X[4][7]$ when the array X is stored in

(a) column major order.

(b) row-major order

Storage for every column = $9 \times 3 = 27\text{B}$

Storage for every row = $20 \times 3 = 60\text{B}$

(a) 7 full-columns and 4 rows come before desired element comes, so address is $3000 + 7 \times 9 \times 3 + 4 \times 3 = 3201\text{B}$

(b) 4 full rows and 7 columns come before desired element, so address $3000 + 4 \times 60 + 7 \times 3 = 3261$

8. (2 points) Find local and global miss rates of L2 and L3 cache from following data.

L1 cache: Accesses 10^6 , misses 700,000

L2 cache: misses 90,000

L3 cache: misses 50,000

L2 cache: Global miss rate = $90000/10^6$

Local miss rate = $90000/700,000$

L3 cache: Global miss rate = $50000/10^6$

Local miss rate = $50000/90,000$

9. (1 point) For address 0xABCD1238, find the set-index in a cache with 16B block and 64 sets (no partial marking). You can leave your answer as binary or decimal value.

Binary is 1010 1011 1100 1101 0001 0010 0011 1000. There are 4 bits for block-offset and set-index has 6 bits, so answer is 10 0011.

10. (3 points) A 32KB cache has a block size of 64B and 8-way associativity. Memory addresses are 32 bits. The cache has valid bits, dirty bits and tags.

(A) Find number of sets in this cache. (1 mark)

(B) In unit of bits, find the number of total bits in this cache (data+valid+dirty+tag). (2 mark)

(A) $32 \times 1024 / (64 \times 8) = 64$ sets

(B) 6 bits for offset and 6 bits for set-index, so 20 bits for tag. So storage for each block is $20\text{tag} + 1\text{valid} + 1\text{dirty} + 512\text{data}$ (in bits) = 534bits

There are $32 \times 1024 / 64$ blocks, so total storage is $(32 \times 1024 / 64) \times 534 = 273408$ bits

11. (7 points) Consider this address pattern, which is seen by a cache. These are byte-addresses.

1, 5, 9, 3, 6, 4, 11, 8, 0, 2, 13, 9

The cache has 2 sets, 2 ways and 2B block size. Classify each of these misses as capacity/compulsory and conflict. Also, show the state of set-0 at the end of this address pattern. Here, you need to show all the ways in this set and all the addresses in each block.

I am showing cache state also; students need not show it. In a set, LRU or MRU are not shown explicitly; that is quite obvious.

Result in fully-associative cache:

A. 1 comes [(0,1), (,), (,),(,)] compulsory miss

B. 5 comes [(0,1), (4,5), (,),(,)] compulsory miss

C. 9 comes [(0,1), (4,5), (8,9),(,)] compulsory miss

D. 3 comes [(0,1), (4,5), (8,9), (2,3)] compulsory miss

E. 6 comes [(6,7), (4,5), (8,9), (2,3)] compulsory miss

F. 4 comes [(6,7), (4,5), (8,9), (2,3)] Hit

G. 11 comes [(6,7), (4,5), (8,9), (10,11)] compulsory miss

H. 8 comes [(6,7), (4,5), (8,9), (10,11)] Hit

I. 0 comes [(0,1), (4,5), (8,9), (10,11)] capacity miss

J. 2 comes [(0,1), (2,3), (8,9), (10,11)] capacity miss

K. 13 comes [(0,1), (2,3), (8,9), (12,13)] compulsory miss

L. 9 comes [(0,1), (2,3), (8,9), (12,13)] Hit

Result in set-associative cache: (different sets are separated by ——)

A. 1 comes [(0,1)—(,)—— (,)—(,)] compulsory miss

B. 5 comes [(0,1)—(4,5)—— (,)—(,)] compulsory miss

C. 9 comes [(8,9)—(4,5)—— (,)—(,)] compulsory miss

D. 3 comes [(8,9)—(4,5)—— (2,3)—(,)] compulsory miss

E. 6 comes [(8,9)—(4,5)—— (2,3)—(6,7)] compulsory miss

F. 4 comes [(8,9)—(4,5)—— (2,3)—(6,7)] Hit

G. 11 comes [(8,9)—(4,5)—— (10,11)—(6,7)] compulsory miss

H. 8 comes [(8,9)—(4,5)—— (10,11)—(6,7)] Hit

I. 0 comes [(8,9)—(0,1)—— (10,11)—(6,7)] Miss; must be capacity

J. 2 comes [(8,9)—(0,1)—— (10,11)—(2,3)] Miss; must be capacity

K. 13 comes [(12,13)—(0,1)—— (10,11)—(2,3)] compulsory miss

L. 9 comes [(12,13)—(8,9)—— (10,11)—(2,3)] miss; must be conflict since it is hit in fully-associative